

Lean 4 中的类型检查

本文档旨在帮助读者更好地理解 Lean 的内核，澄清使用 Lean 时所涉及的信任假设，并为希望为 Lean 的内核语言编写外部类型检查器的读者提供参考资料。

前言

作者谨向下列人士与群体致谢：

Mario Carneiro

Leonardo de Moura

Sebastian Ullrich

Gabriel Ebner

Jonathan Protzenko

[Lean Zulip](#) 社区。

Lean4、Mathlib 以及更广泛的 Lean 生态中的每一位贡献者。

谨以此书纪念 Philip Elliot Bailey（1987 年 1 月 31 日至 2023 年 12 月 9 日）。

什么是内核？

内核是在软件中对 Lean 逻辑的实现：它是一个计算机程序，包含构造 Lean 逻辑语言中的对象并检查这些对象是否正确所需的最小机制。其主要组成部分如下：

- 用于寻址的一类名称。
- 一类宇宙层级。
- 一类表达式（lambda、变量等）。
- 一类声明（公理、定义、定理、归纳类型等）。
- 环境，即从名称到声明的映射。
- 操作表达式的功能。例如，有界变量替换以及宇宙参数替换。
- 类型检查所用的核心操作，包括类型推断、规约和定义相等检查。
- 操作并检查归纳类型声明的功能。例如，生成某个类型的递归器（消去规则），并检查该类型的构造子是否符合类型的规格说明。
- 可选的内核扩展，使上述操作能够作用于自然数和字符串字面量。

将一个小内核隔离出来，并要求 Lean 定义被翻译为极小的内核语言，其目的在于提高证明系统的可信度。Lean 的设计允许用户同一个功能完备的证明助手交互；这个证明助手提供了健壮的元编程、丰富的编辑器支持和可扩展语法等便利功能。同时，它也允许将构造出的证明项抽取为一种形式，使其可以在不信任实现这些高层功能的代码是否正确的前提下得到验证，而这些高层功能正是使 Lean（作为证明助手）高效且宜用的原因。

在 [Certified Programming with Dependent Types](#) 第 1.2.3 节中，Adam Chlipala 定义了通常称为 de Bruijn 准则或 de Bruijn 原则的思想。

当证明助手即便最初使用复杂且可扩展的过程来寻找证明，也仍然在小型内核语言中产生证明项时，它们满足“de Bruijn 准则”。这些核心语言在特性复杂度上与数学形式基础的提案（例如 ZF 集合论）相当。为了相信一个证明，我们可以忽略搜索过程中出现错误的可能性，而只依赖一个（相对较小的）证明检查内核，将它应用于搜索结果。

Lean 的内核足够小，使得开发者可以编写自己的实现，并借助导出器¹独立地检查 Lean 中的证明。Lean 的导出格式包含关于被导出声明的足够信息，因此用户也可以选择只实现完整内核的某些子集。例如，若用户只关心类型推断、规约和定义相等核心功能，就可以选择不实现检查归纳规格说明的功能。

除了上面列出的项目之外，外部类型检查器还需要一个用于 [Lean 导出格式](#) 的解析器，以及一个用于输出的漂亮打印器。解析器和漂亮打印器并不是内核的一部分，但如果希望同内核进行有意义的交互，它们就是重要的组成部分。

¹ 编写自己的类型检查器并不是一个下午就能完成的项目，但它完全属于公民科学家可以实现的范围。

信任

Lean 的核心价值之一，是能够构造数学证明，其中也包括关于程序正确性的证明。用户经常提出的一个问题是：在信任 Lean 时，我们究竟需要信任多少东西，又具体信任什么？

这个问题的答案分为两部分：为了信任 Lean 中的证明，用户需要信任什么；以及为了信任由 Lean 程序编译得到的可执行程序，用户又需要信任什么。

具体而言，二者的区别在于，证明（包括关于程序的命题）以及未编译的程序可以直接用 Lean 的内核语言表示，并由某个内核实现来检查。它们不需要被编译成可执行文件，因此信任范围仅限于用于检查它们的那个内核实现；Lean 编译器不会成为可信代码基的一部分。

信任已编译 Lean 程序的正确性，则要求信任 Lean 编译器；编译器与内核相分离，并不属于 Lean 的核心逻辑。信任 Lean 中的“关于程序的命题”，与信任“由 Lean 编译器产生的程序”，二者并不相同。关于 Lean 程序的命题是证明，属于只需信任内核的范畴。若要信任关于某个程序的证明能够延伸到该程序编译后的行为，就必须将编译器纳入可信代码基。

注意：策略以及其他元程序，即使是已编译的策略，也完全不需要被信任；它们是不可信代码，其作用只是产生供其他部分使用的内核项。一个命题 P 可以在 Lean 中借助任意复杂的已编译元程序得到证明，而无需把可信代码基扩展到内核之外，因为该元程序必须产生一个以 Lean 内核语言表示的证明。

- 这些说法适用于已被[导出](#)的证明。为了满足更为严谨的读者，这当然也意味着还要在一定程度上信任例如运行导出器与验证器的计算机上的操作系统、硬件等。
- 对于未导出的证明，用户还额外信任内核之外的 Lean 组成部分（elaborator、解析器等）。

更细的条目化说明

关于 Lean 4 中涉及的信任，Mario Carneiro 曾在 Lean Zulip 上给出过更细的说明。

一般而言：

1. 你信任 Lean 的逻辑是可靠的（作者注：这也包括任何内核扩展，例如用于 Nat 和 String 的扩展）。
2. 如果你没有证明程序正确，那么你信任 elaborator 已经把你的输入转换成了表示你所期望程序的 Lean 表达式。

3. 如果你已经证明程序正确，那么你信任关于该程序的证明已经被检查过（可使用外部检查器消除这一点）。
4. 你信任运行所有这些东西的硬件、固件和操作系统软件没有损坏，也没有欺骗你。
5. （运行程序时）你信任硬件、固件和操作系统软件按照规格忠实地执行程序，并且没有调试器、硬盘磁铁或宇宙射线干扰你的输出。

对于已编译的可执行文件：

6. 你信任任何编译器覆盖项（extern / implemented_by）没有违反 Lean 逻辑（即模型与实现相匹配）。
7. 你信任 Lean 编译器（它把 Lean 代码降为 C）会保持程序语义。
8. 你信任 clang / LLVM 会把 C 程序转换为具有相同语义的可执行文件。

第一组要点同时适用于证明和已编译的可执行文件；第二组则专门适用于已编译的可执行程序。

外部检查器中的信任

1. 你仍然信任 Lean 的逻辑是可靠的。
2. 你信任外部检查器的开发者正确地实现了这个程序。
3. 你信任实现语言的编译器或解释器。若运行多个外部检查器，可以把它们想象成维恩图中的若干圆：你信任这些圆的交集部分不存在可靠性问题。
4. 对于 Nat 和 String 的内核扩展，你大概还信任一个大整数库，以及实现语言中的 UTF-8 字符串类型。

使用外部检查器的优点包括：

- 用户可以用某个与 Lean 生态完全相互独立、且不依赖 Lean 代码库任何部分的东西来检查自己的结果。
- 外部检查器可以被写成利用成熟的编译器或解释器。
- 对于内核扩展，用户可以交叉检查多个大整数/字符串实现的结果。
- 使用导出功能是摆脱对 Lean 内核之外部分的信任的唯一方式。因此，即便导出文件由 [lean4lean](#) 这样的工具检查，这样做也仍然有价值。因误用 Lean 元编程功能而担心后果的用户，因此应当考虑使用导出功能。

不安全声明

Lean 的交互式语言允许用户编写标记为 `unsafe` 的声明；这类声明可以做通常被禁止的事情。例如，Lean 允许如下定义：

```
unsafe def y : Nat := y
```

不安全声明不会被导出¹，不需要被信任，并且（顺便一提）即使在交互式语言中，也不允许出现在证明中。允许交互式语言包含不安全声明，对 Lean 用户仍然是有益的，因为它让用户在编写用于产生证明、但本身不必是证明的代码时拥有更大的自由度。

aesop 库给出了一个非常好的真实例子。[Aesop](#) 是一个自动化框架；它帮助用户生成证明。在开发过程中的某个时刻，aesop 的作者认为，表达其系统某一部分的最佳方式是使用一个相互定义的归纳类型，[见此处](#)。恰好，这组归纳类型中有一个被声明类型在 Lean 理论中出现的位置是非法的，因此 Lean 内核不允许它；它必须被标记为 `unsafe`。

把这个定义允许为一个 `unsafe` 声明仍然是双赢的。Aesop 的开发者能够用 Lean、按照他们希望的方式编写自己的库，而不必转向（并学习）另一套元编程 DSL；他们也不必为了满足内核而绕许多弯路。与此同时，aesop 的用户仍然可以导出并验证由 aesop 产生的证明，而无需验证 aesop 本身。

¹ 从技术上说，并没有什么阻止一个不安全声明被放入导出文件（尤其是因为导出器不是可信组件）。不过，如果这些不安全声明确实是不安全的，内核运行的检查会阻止它们被加入环境。一个正确实现的类型检查器在收到声明上文所述 aesop 库代码的导出文件时，应当抛出错误。

对抗性输入

常与一般信任问题相伴出现的另一个主题，是 Lean 面对对抗性输入时的健壮性。

正确的类型检查器会按照 Lean 类型系统的规则，并在操作者允许的公理范围内，约束它所接收的输入。若操作者把允许的公理限制为三个“官方”公理（`propext`、`Quot.sound`、`Classical.choice`），那么在任何情况下，输入文件都不应当能够给出一个被类型检查器接受的、关于 prelude 中 `False` 的证明。

然而，一个极小类型检查器不会主动防御那些逻辑上可靠、却被设计来误导人类操作者的 Lean 声明。例如，对手可能重定义他知道审稿人不会检查的深层依赖；或者引入 Unicode 近形字符，使漂亮打印器的输出掩盖关键定义被修改的事实。

“用户可能以为某个定理已经被形式化证明，而事实上他或她误解了系统实际做了什么”这一想法，由 Pollack 一致性问题所刻画，并在 Freek Wiedijk 的这篇论文¹中得到讨论。

请注意，原则上没有任何东西阻止开发者编写软件或扩展类型检查器来防御这类攻击；只是这种保护并不包含在内核所需的最小功能中。不过，Lean 用户对其强大的自定义语法和宏系统的广泛采用，可能会给有意改善这一问题的人带来一些挑战。读者可以把这看作一个[有待未来工作解决的开放问题](#)。

¹ Freek Wiedijk. Pollack-inconsistency. Electronic Notes in Theoretical Computer Science, 285:85-100, 2012

导出格式

导出器是一个程序，它使用内核语言发出 Lean 声明，以供外部类型检查器使用。产生导出文件，意味着完全离开 Lean 生态：文件中的数据可以由完全外部的软件检查，而导出器本身并不是可信组件。与其直接检查导出文件本身，以判断声明是否按照开发者意图被导出，不如由外部检查器检查这些导出的声明，再由漂亮打印器把它们显示给用户；漂亮打印器产生的输出远比导出文件本身更可读。读者可以（也被鼓励）为 Lean 导出文件编写自己的外部检查器。

官方导出器是 [lean4export](#)。

当前版本的 lean4export 使用 ndjson 格式，其规范见[此处](#)。

内核概念

本章先给出一份高层次路线图，所有读者都应阅读；随后介绍理解内核具体机制所需的一些概念，重点不在理论。后几节中的概念不必一开始就完全掌握，因此读者可以先略读，并在需要时再回头查阅。

总体图景

为了给读者一份路线图，检查一个导出文件的整个过程由以下步骤组成：

- 解析导出文件，得到每一种基本类别的组件集合：名称、层级和表达式，以及声明集合。
- 已解析的声明集合表示一个环境，即从每个声明的名称到声明本身的映射；这些声明正是类型检查过程的实际目标。
- 对环境中的每个声明，内核要求：该声明尚未在环境中声明过；它没有重复的宇宙参数；它的类型确实是一个类型而不是一个值（即 `infer declar.ty` 返回某个表达式 `Sort <n>`）；并且该声明的类型没有自由变量。
- 对于定义、定理和不透明声明，断言对定义值进行类型推断得到的表达式，与用户为声明指定的类型定义相等。就断言证明正确性而言，这一步是关键所在；对于定理，它对应于这样的问题：“用户声称这是 `P` 的一个证明，那么这个值实际上是否构成了 `P` 的有效证明？”
- 对于归纳声明、其构造子和递归器，检查它们是否形式良好，并符合 Lean 类型论的规则（后文会进一步说明）。
- 如果导出文件包含商类型的原始声明，则确保这些声明具有正确的类型，并确保 `Eq` 类型存在且定义正确（因为商类型依赖等式）。
- 最后，漂亮打印用户请求的任何声明，使用户能够检查被检查的声明是否与他们导出的声明相匹配。

术语说明

Type、Sort、Prop

`Prop` 指 `Sort 0`。

`Type n` 指 `Sort (n+1)`。

`Sort n` 是它们在内核中的实际表示，并且总是可以直接使用。

有时使用 `Type <N>` 和 `Prop`，而不是始终坚持写成 `Sort n`，是因为 `Type <N>` 的元素具有某些重要性质和行为，而这些性质和行为并不会出现在 `Prop` 的元素中；反之亦然。

例如：只有 `Prop` 的元素可以被纳入定义上的证明无关性考虑；而只有 `Type _` 的元素可以在不满足其他测试的情况下使用大消去。

层级/宇宙与 Sort

术语“层级”(level) 和“宇宙”(universe) 基本同义；它们指同一种内核对象。

有时会作一个小区分：“宇宙参数”可能被隐含地限制为变量/参数形式的层级。这是因为“宇宙参数”指的是参数化某个 Lean 声明的一组层级；这些层级只能是标识符，因此也被限制为标识符。如果这句话暂时没有意义，不必担心。举例来说，一个 Lean 声明可以以 `def Foo.{u} ..` 开头，意思是“一个由宇宙变量 `u` 参数化的定义”；但它不能以 `def Foo.{1} ..` 开头，因为 `1` 是一个显式层级，而不是参数。

另一方面，`Sort _` 是一个表示层级的表达式。

值与类型

表达式既可以是值，也可以是类型。读者大概熟悉这样的观念：`Nat.zero` 是值，而 `Nat` 是类型。若 `infer e != Sort _`，则表达式 `e` 是一个值，或者说是“值层级表达式”。若 `infer(e) = Sort _`，则表达式 `e` 是一个类型，或者说是“类型层级表达式”。

参数与指标

参数（parameter）与指标（index）的区别，会在处理归纳类型时起作用。粗略地说，声明中冒号之前的望远镜元素是参数，冒号之后的元素是指标：

```
parameter ----v          v---- index
inductive Nat.le (n : Nat) : Nat → Prop
```

这个区别在内核中不可忽略，因为参数在一个声明内部是固定的，而指标则不是。

实例化与抽象

实例化指的是用适当的实参替换有界变量。抽象指的是在重建绑定子时，用适当的有界变量替换自由变量。Lean 的内核对有界变量使用 de Bruijn 指标，对自由变量使用唯一标识符。

就本文而言，自由变量是表达式中的一个变量，它指向某个已经被“打开”、因而不立即可用的绑定子；因此，我们用一个自由变量替换相应的有界变量，并在这个自由变量上保存一些关于被留下的绑定子的资料。

为了说明这一点，假设有某个 lambda 表达式 `(fun (x : A) => <body>)`，并且我们正在进行类型推断。类型推断必须遍历进入表达式的 `<body>` 部分，而其中可能包含一个指向 `x` 的有界变量。当我们进入函数体时，可以把 `x` 加入某个有状态的绑定子上下文，并把整个上下文带入函数体；也可以临时把所有指向 `x` 的有界变量替换为一个自由变量，从而无需携带额外上下文就能进入函数体。

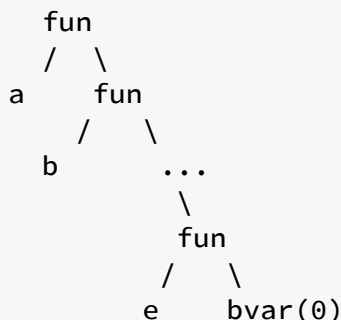
如果我们最终回到打开绑定子之前的位置，抽象就允许我们把所有那些曾经是指向 `x` 的有界变量的自由变量，替换为新的有界变量，使它们再次指向 `x`，并具有正确的 de Bruijn 指标。

实现自由变量抽象

对于 de Bruijn 层级，自由变量会记录一个数字，其含义是：“我是表示从望远镜顶部数第 `n` 个有界变量的自由变量”。

这与 de Bruijn 指标相反；de Bruijn 指标中的数字表示“从望远镜底部数第 `n` 个有界变量”。

这里的顶部和底部，是把表达式的望远镜想象成一棵树时的说法：



例如，对于 lambda 表达式 `fun (a b c d e) => bvar(0)`，这个有界变量通过引用“从底部数第 0 个”而指向 `e`。

在 lambda 表达式 `fun (a b c d e) => fvar(4)` 中，这个自由变量也是表示 `e` 的 de Bruijn 层级，但这一次它表示的是“从望远镜顶部数第 4 个”。

为什么要作这种区分？当我们在强规约过程中创建自由变量时，我们知道几件事：我们知道即将替换进去的自由变量可能会被后续规约移动；我们知道自己上方有多少个已打开的绑定子（因为我们必须访问它们才能到达这里）；我们也知道，为了重新替换绑定子，我们之后可能需要引用/抽象这个表达式，也就是说，需要重新绑定这个自由变量。然而，在那个时刻，我们并不知道我们下方还剩多少绑定子，因此不能说这个变量在最终被抽象/引用时会是从底部数第几个变量。

对于使用唯一标识符标记自由变量的实现，这个问题通过在抽象过程中持有正在重建的实际望远镜来解决。只要有表达式和一份唯一标记自由变量的列表，就可以进行抽象，因为自由变量在列表中的位置表明了它们的绑定子位置。

弱规约与强规约

Lean 规约策略的实现细节将在[另一章](#)讨论；本节专门说明弱规约与强规约之间的区别。

弱规约

弱规约指的是在遇到没有施加实参的绑定子时停止的规约。这里的绑定子指 lambda、pi 和 let 表达式（不过在当前语境中，我们只需关注 lambda）。

例如，弱规约可以把 `(fun (x y : Nat) => y + x) (0 : Nat)` 规约为 `(fun (y : Nat) => y + 0)`，但无法再继续作进一步的（弱）规约。

当我们说“弱头范式规约”，或者只说“规约”而没有特别说明它是“强”的时候，说的就是弱规约。强规约只是我们在其他地方打开某个绑定子后，再应用弱规约时产生的副产物。

强规约

强规约指的是在打开的绑定子之下进行规约：当我们遇到一个没有伴随实参的绑定子（例如一个没有 `app` 节点为其施加实参的 lambda 表达式）时，可以遍历进入其函数体，并通过创建并替换一个自由变量来继续规约。类型推断和定义相等检查都需要强规约。对于类型推断，我们还需要能够“重新闭合”打开的项：在函数体中完成若干规约后，用正确的有界变量替换自由变量。这并不是简单地换回原先同一个有界变量即可，因为有界变量可能已经发生位移，使旧的 de Bruijn 指标相对于新的重建表达式失效。

与弱规约一样，强规约也可以把 `(fun (x y : Nat) => y + x) (0 : Nat)` 规约为 `(fun (y : Nat) => y + 0)`；但它不会在这里卡住，而是可以继续用一个自由变量替换 `y`，把表达式进一步规约为 `((fVar id, y, Nat) + 0)`，再规约为 `(fvar id, y, Nat)`。

只要我们在某处保留自由变量信息，就可以把这些信息与规约后的 `(fVar id, y, Nat)` 重新组合起来，重建出 `(fun (y : Nat) => bvar(0))`。

名称

内核的第一个原始类型是 `Name`，大体上正如其名；它为内核项提供一种寻址事物的方式。

```
Name ::= anonymous | str Name String | num Name Nat
```

`Name` 类型的元素会显示为以点分隔的名称，Lean 用户大概对此很熟悉。例如，`num (str (anonymous) "foo") 7` 会显示为 `foo.7`。

实现说明

名称的实现假定字符串为 UTF-8，字符为 Unicode 标量值（关于实现语言字符串类型的这些假设，对于字符串字面量内核扩展也很重要）。

关于名称词法结构的一些信息见[此处](#)。

导出器不会显式输出匿名名称，而是期望它是导入名称中的第 0 个元素。

宇宙层级

本节将从实现角度描述宇宙层级，并说明读者在对 Lean 声明进行类型检查时需要知道的内容。关于宇宙层级在 Lean 类型论中的作用，更深入的讨论可见 [TPIL4](#)，或 [Mario Carneiro 的博士论文](#) 第 2 节。

宇宙层级的语法如下：

```
Level ::= Zero | Succ Level | Max Level Level | IMax Level Level | Param Name
```

读者应当注意 `Level` 类型的几个性质：宇宙层级上存在一个偏序；其中存在变量（`Param` 构造子）；以及 `Max` 与 `IMax` 之间的区别。

`Max` 只是构造一个表示左右两个参数中较大者的宇宙层级。例如，`Max(1, 2)` 化简为 `2`，而 `Max(u, u+1)` 化简为 `u+1`。`IMax` 构造子也表示左右两个参数中的较大者，除非右参数化简为 `Zero`；在这种情况下，整个 `IMax` 解析为 `0`。

`IMax` 的重要之处在于它与类型推断过程的相互作用。例如，它确保 `forall (x y : Sort 3), Nat` 被推断为 `Sort 4`，而 `forall (x y : Sort 3), True` 被推断为 `Prop`。

层级上的偏序

Lean 的 `Level` 类型带有一个偏序，也就是说，我们可以对一对层级执行“小于等于”测试。下面这个相当漂亮的实现来自 Gabriel Ebner 的 Lean 3 检查器 [trepplein](#)。虽然需要覆盖的情形不少，但唯一复杂的匹配是依赖 `cases` 的那些；`cases` 通过分别检查把参数 `p` 替换为 `Zero` 时，以及把 `p` 替换为 `Succ p` 时， `$x \leq y$` 是否成立，来判断 `$x \leq y$` 。

```

leq (x y : Level) (balance : Integer): bool :=
  Zero, _ if balance >= 0 => true
  _, Zero if balance < 0 => false
  Param(i), Param(j) => i == j && balance >= 0
  Param(_), Zero => false
  Zero, Param(_) => balance >= 0
  Succ(l1_), _ => leq l1_ l2 (balance - 1)
  _, Succ(l2_) => leq l1 l2_ (balance + 1)

-- 向左下降
Max(a, b), _ => (leq a l2 balance) && (leq b l2 balance)

-- 向右下降
(Parm(_) | Zero), Max(a, b) => (leq l1 a balance) || (leq l1 b balance)

-- imax
IMax(_, p @ Param(_)), _ => cases(p)
_, IMax(_, p @ Param(_)) => cases(p)
IMax(a1, b1), IMax(a2, b2) if a1 == a2 && b1 == b2 && balance >= 0 => true
IMax(a, IMax(b, c)), _ => leq Max(IMax(a, c), IMax(b, c)) l2 balance
IMax(a, Max(b, c)), _ => leq (simplify Max(IMax(a, b), IMax(a, c))) l2 balance
_, IMax(a, IMax(b, c)) => leq l1 Max(IMax(a, c), IMax(b, c)) balance
_, IMax(a, Max(b, c)) => leq l1 (simplify Max(IMax(a, b), IMax(a, c))) balance

cases l1 l2 p: bool :=
  leq (simplify $ subst l1 p zero) (simplify $ subst l2 p zero)
  ^
  leq (simplify $ subst l1 p (Succ p)) (simplify $ subst l2 p (Succ p))

```

层级相等

`Level` 类型通过反对称性识别相等：两个层级 `l1` 和 `l2` 相等，当且仅当 `l1 ≤ l2` 且 `l2 ≤ l1`。

实现说明

注意，导出器不会导出 `zero`；它被假定为 `Level` 的第 0 个元素。

顺便说一句，`Level` 的实现对性能没有很大影响，因此不必在这里过度优化。

表达式

完整语法

下面会更详细地解释表达式。不过为了先把全貌摆出来，包含字符串和自然数字面量扩展在内的表达式完整语法如下：

```
Expr ::=
  | boundVariable
  | freeVariable
  | const
  | sort
  | app
  | lambda
  | forall
  | let
  | proj
  | literal

BinderInfo ::= Default | Implicit | InstanceImplicit | StrictImplicit

const ::= Name, Level*
sort ::= Level
app ::= Expr Expr
-- 一个 deBruijn 指标
boundVariable ::= Nat
lambda ::= Name, (binderType : Expr), BinderInfo, (body : Expr)
forall ::= Name, (binderType : Expr), BinderInfo, (body : Expr)
let ::= Name, (binderType : Expr), (val : Expr) (body : Expr)
proj ::= Name Nat Expr
literal ::= natLit | stringLit

-- 任意精度自然数/无符号整数
natLit ::= Nat
-- UTF-8 字符串
stringLit ::= String

-- fvarId 可以用唯一名称或 deBruijn 层级实现；
-- 唯一名称更灵活，deBruijn 层级有更好的缓存行为。
freeVariable ::= Name, Expr, BinderInfo, fvarId
```

几点说明：

- 自然数字面量所用的 `Nat` 应当是任意精度自然数/大整数。

- 带有绑定子的表达式（`lambda`、`pi`、`let`、自由变量）也完全可以把三个参数（绑定子名称、绑定子类型、绑定子样式）打包成一个参数 `Binder`，其中绑定子为 `Binder ::= Name BinderInfo Expr`。在别处出现的伪代码中，我通常会像它们具有这种性质一样处理，因为这样更易读。
- 自由变量标识符可以是唯一标识符，也可以是 de Bruijn 层级。
- Lean 本身使用的表达式类型还有一个 `mdata` 构造子，用来声明附带元数据的表达式。该元数据不影响表达式在内核中的行为，因此这里不包含这个构造子。

绑定子信息

用 `lambda`、`pi`、`let` 和自由变量构造子构造的表达式包含绑定子信息，其形式为一个名称、一个绑定子“样式”和绑定子的类型。绑定子的名称与样式只供漂亮打印器使用，不会改变推断、规约或相等性检查的核心过程。不过在漂亮打印器中，绑定子样式可能会根据漂亮打印选项改变输出。例如，用户可能想显示，也可能不想显示隐式参数或实例隐式参数（类型类变量）。

Sort

`sort` 只是对层级的一层包装，使其可以作为表达式使用。

有界变量

有界变量被实现为表示 [de Bruijn 指标](#)的自然数。

自由变量

自由变量用于在绑定子当前不可用的情形下传达关于有界变量的信息。通常，这是因为内核已经遍历进入某个绑定表达式的函数体，并选择不携带一份结构化的绑定信息上下文，而是临时把有界变量替换为一个自由变量；之后可以再换入一个新的（也许不同的）有界变量来重建绑定子。这种“不可用”的描述可能听起来有些含糊，但一个或许有帮助的直观解释是：表达式被实现为没有任何父指针的树，因此当我们下降到子节点（尤其是跨越函数边界）时，就会失去对当前表达式中位于我们上方那些元素的视野。

当通过重建绑定子来闭合一个打开表达式时，绑定关系可能已经改变，从而使先前有效的 de Bruijn 指标失效。使用唯一名称或 de Bruijn 层级，可以用一种补偿这些变化的方式重新闭合绑定子，并确保重新绑定变量的新 de Bruijn 指标相对于重建后的望远镜是有效的（见[本节](#)）。

下文中，我们可能会用某种形式的“自由变量标识符”一词，来指称某个实现所采用方案（唯一 ID 或 de Bruijn 层级）中的对象。

Const

`const` 构造子是表达式引用环境中另一个声明的方式；它必须通过引用做到这一点。

在下面的例子中，`def plusOne` 创建了一个 `Definition` 声明，该声明被检查后进入环境。声明不能直接放入表达式中，因此当 `plusOne_eq_succ` 的类型调用先前的声明 `plusOne` 时，它必须通过名称来调用。于是创建表达式 `Expr.const (plusOne, [])`；当内核发现这个 `const` 表达式时，它可以在环境中按名称查找被引用的声明 `plusOne`：

```
def plusOne : Nat -> Nat := fun x => x + 1

theorem plusOne_eq_succ (n : Nat) : plusOne n = n.succ := rfl
```

用 `const` 构造子创建的表达式还携带一组层级。通过查找该 `const` 表达式所引用的定义而从环境中取出的任何展开声明或推断声明，都会把这些层级替换进去。例如，对 `const List [Level.param(x)]` 推断类型时，要在当前环境中查找 `List` 的声明，取出它的类型和宇宙参数，然后把 `x` 替换为 `List` 最初声明时使用的宇宙参数。

Lambda、Pi

`lambda` 和 `pi` 表达式（Lean 本身使用名称 `forallE` 而不是 `pi`）分别指函数抽象和“forall”绑定子（依赖函数类型）。

```
      binderName      body
        |              |
fun (foo : Bar) => 0
        |
      binderType
```

-- `BinderInfo` 通过包围绑定子所用括号的样式体现。

Let

`let` 正如其名。虽然 `let` 表达式也是绑定子，但它们没有 `BinderInfo`；其绑定子信息被视为 `Default`。

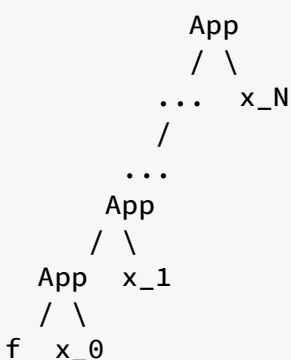
```

      binderName      val
        |             |
let (foo : Bar) := 0; foo
        |             |
      binderType      .... body

```

App

`app` 表达式表示把一个实参应用到一个函数。App 节点是二叉的（只有两个子节点：一个函数和一个实参），因此 `f x_0 x_1 .. x_N` 会被表示为 `App(App(App(f, x_0), x_1)..., x_N)`，也可以可视化如下树：



操作表达式时，一个极其常见的内核操作，是折叠和展开一串应用：从上面的树结构得到 `(f, [x_0, x_1, .., x_N])`，或者把 `f, [x_0, x_1, .., x_N]` 折叠回上面的树。

投影

`proj` 构造子表示结构投影。只有一个构造子且没有指标的归纳类型可以作为结构。

这个构造子接受一个名称（类型的名称）、一个表示被投影字段的自然数，以及实际被投影的结构。

注意，在内核中，投影指标从 0 开始；而在 Lean 的交互式语言中它从 1 开始。这里的 0 表示跳过参数后的第一个字段。

例如，内核表达式 `proj Prod 0 (@Prod.mk A B a b)` 会投影出 `a`，因为跳过参数 `A` 和 `B` 后，它是第 0 个字段。

虽然 `proj` 所提供的行为也可以通过使用该类型的递归器实现，但 `proj` 能更高效地处理频繁出现的内核操作。

字面量

Lean 的内核可选地支持任意精度 Nat 和 String 字面量。必要时，内核可以把自然数字面量 `n` 转换为 `Nat.zero` 或 `Nat.succ m`，也可以把字符串字面量 `s` 转换为 `String.mk List.nil` 或 `String.mk (List.cons (Char.ofNat _) ...)`。

字符串字面量会在测试定义相等时，以及在作为递归器规约中的主前提出现时，被惰性地转换为字符列表。

自然数字面量在与字符串相同的位置得到支持（定义相等以及递归器应用的主前提），但内核还支持对自然数字面量进行加法、乘法、幂、减法、取模、除法，以及布尔相等和“小于等于”比较。

实现表达式

对于有意全部或部分编写自己内核的读者，表达式类型有几个值得注意的要点.....

存储的数据

表达式需要在内部存储一些数据，或在某处缓存这些数据，以避免代价过高的重复计算。例如，创建表达式 `app x y` 时，需要计算并存储所得 `app` 表达式的哈希摘要；而这样做应当通过获取 `x` 和 `y` 已缓存的哈希摘要来完成，而不是遍历 `x` 的整棵树递归计算 `x` 的摘要，再对 `y` 做同样的事。

你大概会希望内联存储的数据包括：哈希摘要、表达式中的松散有界变量数量，以及表达式是否含有自由变量。后两者分别有助于优化实例化和抽象。

一个用于 `app` 表达式的“智能构造器”示例如下：

```
def mkApp x y:
  let hash := hash x.storedHash y.storedHash
  let numLooseBvars := max x.numLooseBvars y.numLooseBvars
  let hasFvars := x.hasFvars || y.hasFvars
  .app x y (cachedData := hash numLooseBvars hasFvars)
```

不作深拷贝

表达式的实现应当使得用于构造某个父表达式的子表达式不会被深拷贝。换言之，创建表达式 `app x y` 不应递归复制 `x` 和 `y` 的元素，而应取得某种引用：可以是指针、整数索引、对垃圾回收对象的引用、引用计数对象，或其他形式（这些策略都应能提供可接受的性能）。如果你构造表达式的默认策略涉及深拷贝，那么在不消耗巨量内存的情况下，将无法构造任何非平凡环境。

松散有界变量数量的示例实现

```
numLooseBVars e:
  match e with
  | Sort | Const | FVar | StringLit | NatLit => 0
  | Var dbjIdx => dbjIdx + 1,
  | App fun arg => max fun.numLooseBVars arg.numLooseBVars
  | Pi binder body | Lambda binder body =>
    max binder.numLooseBVars (body.numLooseBVars - 1)
  | Let binder val body =>
    max (max binder.numLooseBVars val.numLooseBVars) (body.numLooseBVars - 1)
  | Proj _ _ structure => structure.numLooseBVars
```

对于 `Var` 表达式，松散有界变量的数量是 de Bruijn 指标加一，因为我们计算的是为了使该变量不再松散而需要放在该变量上方的绑定子数量（这里的 `+1` 是因为 de Bruijn 指标从 0 开始）。对于表达式 `Var(0)`，需要在有界变量上方放置一个绑定子，才能使该变量不再松散。对于 `Var(3)`，则需要四个：

```
-- 3 2 1 0
fun a b c d => Var(3)
```

当我们创建一个新的绑定子（`lambda`、`pi` 或 `let`）时，可以从函数体中的松散 `bvar` 数量中减去 1（使用饱和/自然数减法），因为函数体现在位于一个额外的绑定子之下。

声明

声明是最重要的对象，也是我们还需要定义的最后一类领域元素。

```
declarationInfo ::= Name, (universe params : List Level), (type : Expr)

declar ::=
  Axiom declarationInfo
  | Definition declarationInfo (value : Expr) ReducibilityHint
  | Theorem declarationInfo (value : Expr)
  | Opaque declarationInfo (value : Expr)
  | Quot declarationInfo
  | InductiveType
    declarationInfo
    is_recursive: Bool
    num_params: Nat
    num_indices: Nat
    -- 此类型以及同一 mutual 块中任何其他类型的名称
    allIndNames: Name+
    -- 仅属于“此”类型的构造子名称，
    -- 不包括同一块中 mutual 类型的构造子。
    constructorNames: Name*

  | Constructor
    declarationInfo
    (inductiveName : Name)
    (numParams : Nat)
    (numFields : Nat)

  | Recursor
    declarationInfo
    numParams : Nat
    numIndices : Nat
    numMotives : Nat
    numMinors : Nat
    RecRule+
    isK : Bool

RecRule ::= (constructor name : Name), (number of constructor args : Nat), (val : Expr)
```

检查声明

对于所有声明，在执行任何针对特定声明种类的额外过程之前，都会先进行下列初步检查：

- 声明的 `declarationInfo` 中的宇宙参数不得有重复。例如，声明 `def Foo.{u, v, u} ...` 会被禁止。
- 声明的类型不得含有自由变量；“完成”的声明中的所有变量都必须对应于某个绑定子。
- 声明的类型必须是一个类型（`infer declarationInfo.type` 必须产生一个 `Sort`）。在 Lean 中，不允许声明 `def Foo : Nat.succ := .. ; Nat.succ` 是值，不是类型。

公理

对公理所做的检查，只有适用于所有声明的那些检查，即确保 `declarationInfo` 合格。如果一个公理具有有效的一组宇宙参数，并且具有一个没有自由变量的有效类型，它就会被接纳进环境。

Quot

`Quot` 声明包括 `Quot`、`Quot.mk`、`Quot.ind` 和 `Quot.lift`。这些声明具有规定好的类型，并且这些类型在 Lean 理论中已知是可靠的；因此环境中的商类型声明必须与这些类型完全匹配。由于这些类型并不复杂到不可接受，内核实现通常会将它们硬编码。

定义、定理、不透明声明

定义、定理和不透明声明的有趣之处在于，它们同时具有类型和值。检查这些声明时，要先对声明的值推断出一个类型，然后断言该推断类型与 `declarationInfo` 中标注的类型定义相等。

在定理的情形中，`declarationInfo` 中的类型是用户声称的类型，因此也是用户声称要证明的命题；而值则是用户为该类型提供的证明。对收到的值推断类型，相当于检查该证明实际上证明了什么；定义相等断言则确保该值所证明的东西，确实是用户打算证明的东西。

可规约性提示

可规约性提示包含关于某个声明应如何展开的信息。`abbreviation` 通常总是会被展开，`opaque` 不会被展开，而 `regular N` 可能会根据 `N` 的值被展开。`regular` 可规约性提示对应于定义的“高度”，也就是该定义用来定义自身所使用的声明数量。若定义 `x` 的值引用了定义 `y`，则 `x` 的高度值会大于 `y`。

归纳类型的秘密生活

归纳声明

为求清楚，“一个归纳声明”的全貌包含一个类型、一个构造子列表，以及一个递归器列表。声明的类型和构造子由用户指定，而递归器则从这些元素派生出来。每个递归器还会得到一组“递归器规则”，也称为计算规则；它们是用于 `iota` 规约（模式匹配的一个较华丽说法）的值层级表达式。下文中，我们会尽力区分“一个归纳类型”和“一个归纳声明”。

Lean 的内核原生支持相互归纳声明；在这种情况下，会有一个由（类型、构造子列表）对组成的列表。内核通过临时把嵌套归纳声明转换为相互归纳声明来支持它们（下面会进一步说明）。

归纳类型

内核要求归纳声明中的“归纳类型”部分确实是一个类型，而不是一个值（`infer ty` 必须产生某个 `sort <n>`）。对于相互归纳声明，被声明的各个类型必须处于同一宇宙，并具有相同的参数。

构造子

对于归纳类型的任意构造子，内核会强制执行以下检查：

- 构造子的类型/望远镜必须与被声明归纳类型的类型共享相同参数。
- 对于构造子类型的望远镜中的非参数元素，其绑定子类型必须确实是一个类型（必须推断为 `Sort _`）。
- 对于构造子类型的望远镜中的任何非参数元素，该元素推断出的 `sort` 必须小于等于归纳类型的 `sort`；否则，被声明的归纳类型必须是一个命题。
- 构造子的任何实参都不得包含被声明类型的非正出现（读者可在[此处](#)深入了解这个问题）。
- 构造子望远镜的末端必须是若干实参对被声明类型的有效应用。例如，我们要求 `List.cons ..` 构造子以 `.. -> List A` 结束；若 `List.cons` 以 `.. -> Nat` 结束，则是错误。

嵌套归纳类型

检查嵌套归纳类型是一个更繁琐的过程：它会临时特化相互块中归纳类型的嵌套部分，使我们只需面对一组“普通的”（非嵌套的）相互归纳类型；然后检查这些特化后的类型；最后再将所有东西去特化，

并接纳这些类型。

考虑如下 S-表达式定义，其中包含嵌套构造 `Array Sexpr`：

```
inductive Sexpr
| atom (c : Char) : Sexpr
| ofArray : Array Sexpr -> Sexpr
```

放大来看，检查一个嵌套归纳声明的过程有三个步骤：

1. 通过特化当前类型所嵌入的“容器类型”，把嵌套归纳声明转换为相互归纳声明。如果容器类型本身又根据其他类型定义，我们也需要触及这些组成部分以便特化。在上例中，我们把 `Array` 用作容器类型，而 `Array` 又根据 `List` 定义，因此需要同时把 `Array` 和 `List` 当作容器类型处理。
2. 对相互归纳类型执行通常的检查和构造步骤。
3. 把特化后的嵌套类型转换回原始形式（去特化），并把恢复/去特化后的声明加入环境。

这种特化的一个例子，是把上面的 `Sexpr` 嵌套归纳声明转换为：

```
mutual
  inductive Sexpr
    | atom : Char -> Sexpr
    | ofList : ListSexpr -> Sexpr

  inductive ListSexpr
    | nil : ListSexpr
    | cons : Sexpr -> ListSexpr -> ListSexpr

  inductive ArraySexpr
    | mk : ListSexpr -> ArraySexpr
end
```

随后在检查这些类型的过程中恢复原来的归纳声明。为澄清起见，当我们说“特化”时，上面的新类型 `ListSexpr` 和 `ArraySexpr` 是在这样的意义上被特化的：它们只被定义为 `Sexpr` 的列表和数组，而不像通常的 `List` 类型那样对某个任意类型保持泛型。

递归器

暂时请参阅[规约一章中关于 `iota` 规约的部分](#)。

类型推断

类型推断是确定给定表达式之类型的过程，也是 Lean 内核的核心功能之一。我们正是通过类型推断确定 `Nat.zero` 是类型 `Nat` 的一个元素，或者 `(fun (x : Char) => var(0))` 是类型 `Char -> Char` 的一个元素。

本节先考察最简单的完整类型推断过程，再考察各过程性能更好、但略微更复杂的版本。

我们还会看到 Lean 内核在类型推断期间作出的若干额外正确性断言。

有界变量

如果你遵循 Lean 的实现并使用局部无名（locally nameless）方法，那么在类型推断期间不应遇到有界变量，因为所有打开的绑定子都会用适当的自由变量实例化。

当遇到绑定子时，我们需要遍历进入其函数体，以确定函数体的类型。为了保留关于绑定子类型的信息，有两种主要做法。Lean 本身使用的是：创建一个保留绑定子类型信息的自由变量，使用实例化把相应有界变量替换为这个自由变量，然后进入函数体。这样做很好，因为我们不必为类型上下文额外维护一份状态。

对于基于闭包的实现，通常会有一个单独的类型上下文，用来记录打开的绑定子；此时遇到有界变量，就意味着要索引进入类型上下文，以取得该变量的类型。

自由变量

创建自由变量时，它会得到所表示绑定子的类型信息，因此我们可以直接把该类型信息作为推断结果。

```
infer FVar id binder:  
  binder.type
```

函数应用

```
infer App(f, arg):  
  match (whnf $ infer f) with  
  | Pi binder body =>  
    assert! defEq(binder.type, infer arg)  
    instantiate(body, arg)  
  | _ => error
```

这里需要的额外断言是：`arg` 的类型与 `binder` 的类型相匹配。例如，在表达式

```
(fun (n : Nat) => 2 * n) 10
```

中，我们需要断言 `defEq(Nat, infer(10))`。

虽然现有实现倾向于在线执行这一检查，但原则上也可以把这个相等性断言存储起来，在别处处理。

Lambda

```
infer Lambda(binder, body):  
  assert! infersAsSort(binder.type)  
  let binderFvar := fvar(binder)  
  let bodyType := infer $ instantiate(body, binderFvar)  
  Pi binder (abstract bodyType binderFvar)
```

Pi

```
infer Pi binder body:  
  let l := inferSortOf binder  
  let r := inferSortOf $ instantiate body (fvar(binder))  
  imax(l, r)  
  
inferSortOf e:  
  match (whnf (infer e)) with  
  | sort level => level  
  | _ => error
```

Sort

任意 `Sort n` 的类型就是 `Sort (n+1)`。

```
infer Sort level:
  Sort (succ level)
```

Const

`const` 表达式用于按名称引用其他声明；被引用的任何其他声明都必须已经事先声明并经过类型检查。因此，我们已经知道被引用声明的类型，只需在环境中查找它即可。不过，我们确实必须把当前声明的宇宙层级替换进被索引定义的宇宙参数中。

```
infer Const name levels:
  let knownType := environment[name].type
  substituteLevels (e := knownType) (ks := knownType.uparams) (vs := levels)
```

Let

```
infer Let binder val body:
  assert! inferSortOf binder
  assert! defEq(infer(val), binder.type)
  infer (instantiate body val)
```

Proj

我们试图推断类似 `Proj (projIdx := 0) (structure := Prod.mk A B (a : A) (b : B))` 这样的东西的类型。

首先推断所给结构的类型；由此可以得到结构名称，并在环境中查找该结构及其构造子类型。

遍历构造子类型的望远镜，把 `Prod.mk` 的参数替换进构造子类型的望远镜中。若查得构造子类型为

`A -> B -> (a : A) -> (b : B) -> Prod A B`，则替换 `A` 和 `B`，留下望远镜 `(a : A) -> (b : B) -> Prod A B`。

构造子望远镜剩余的部分表示结构字段，其类型信息位于绑定子中，因此可以检查

`telescope[projIdx]` 并取出绑定子类型。不过还需要处理一件事：由于靠后的结构字段可能依赖靠

前的结构字段，我们需要用 `proj thisFieldIdx s` 来实例化望远镜的其余部分（每一阶段的函数体），其中 `s` 是我们正在推断的投影表达式中的原始结构。

```
infer Projection(projIdx, structure):
  let structType := whnf (infer structure)
  let (const structTyName levels) tyArgs := structType.unfoldApps
  let InductiveInfo := env[structTyName]
  -- 这个归纳类型既然声称是结构，就应当只有一个构造子。
  let ConstructorInfo := env[InductiveInfo.constructorNames[0]]

  let mut constructorType := substLevels ConstructorInfo.type (newLevels :=
levels)

  for tyArg in tyArgs.take constructorType.numParams
    match (whnf constructorType) with
    | pi _ body => inst body tyArg
    | _ => error

  for i in [0:projIdx]
    match (whnf constructorType) with
    | pi _ body => inst body (proj i structure)
    | _ => error

  match (whnf constructorType) with
  | pi binder _ => binder.type
  | _ => error
```

自然数字面量

自然数字面量推断为指向声明 `Nat` 的常量。

```
infer NatLiteral _:
  Const(Nat, [])
```

字符串字面量

字符串字面量推断为指向声明 `String` 的常量。

```
infer StringLiteral _:
  Const(String, [])
```

定义相等

定义相等被实现为一个函数：它接受两个表达式作为输入；若这两个表达式在 Lean 理论中定义相等，则返回 `true`，否则返回 `false`。

在内核内部，定义相等之所以重要，只是因为它是类型检查的必要组成部分。对于不深入内核的 Lean 用户而言，定义相等仍然是一个重要概念，因为在 Lean 的交互式语言中，定义相等比较容易使用：对于任意定义相等的 `a` 和 `b`，Lean 不需要用户提示或提供额外输入，就能判定两个表达式相等。

实现定义相等过程有两个宏观部分。第一部分是用于检查不同定义相等性的各个具体测试。对于只想从终端用户角度理解定义相等的读者，这大概就是你想知道的内容。

有意编写类型检查器的读者还应理解这些具体检查如何同规约和缓存组合起来，使问题可处理；天真地逐个运行检查并在途中规约，可能会得到不可接受的性能结果。关于这一点，我们建议参考已有的内核实现。**这些检查执行的顺序和方式会显著影响性能**；虽然像 Mathlib 这样较大的 Lean 项目可能试图在内核中使用和滥用规约与相等性检查方面保持某种纪律，但代码总会不可避免地依赖某些内核执行路径在 C++ 内核实现中很快或具有某种优势。因此，若用替代策略或更天真的策略检查这些代码库，很可能会造成性能问题。

语法相等（也称结构相等或指针相等）

只要类型检查器确保两个对象相等当且仅当它们由相同组件构造而来（相关构造子是 `Name`、`Level` 和 `Expr` 的构造子），那么当两个表达式指向完全相同的实现对象时，它们就是定义相等的。

Sort 相等

两个 `Sort` 表达式定义相等，当且仅当它们表示的层级根据层级偏序的反对称性相等。

```
defEq (Sort x) (Sort y):  
  x ≤ y ∧ y ≤ x
```

Const 相等

两个 `Const` 表达式定义相等，当且仅当它们的名称相同，并且它们的层级在反对称意义下相等。

```
defEq (Const n xs) (Const m ys):  
  n == m  $\wedge$  forall (x, y) in (xs, ys), antisymmEq x y  
  
-- 如果你的 `zip` 不会检查长度，也要断言 xs 和 ys 长度相同。
```

有界变量

对于使用基于替换的策略（如局部无名）的实现而言（如果你遵循 C++ 或 lean4lean 实现，这说的就是你），遇到有界变量是一个错误；如果有界变量指向一个实参，那么它应当已经在弱规约期间被替换；或者它应当已经在对 pi 或 lambda 表达式作定义相等检查时，作为强规约的一部分被替换为自由变量。

对于基于闭包的实现，则查找与有界变量对应的元素，并断言它们定义相等。

自由变量

两个自由变量定义相等，当且仅当它们具有相同的标识符（唯一 ID 或 de Bruijn 层级）。关于绑定子类型相等的断言，应当已经在构造自由变量的地方执行过（例如 pi/lambda 表达式的定义相等检查），因此现在不必重新检查。

```
defEqFVar (id1, _) (id2, _):  
  id1 == id2
```

App

两个应用表达式定义相等，当且仅当它们的函数部分和实参部分分别定义相等。

```
defEqApp (App f a) (App g b):  
  defEq f g  $\wedge$  defEq a b
```

Pi

两个 Pi 表达式定义相等，当且仅当它们的绑定子类型定义相等，并且在替换进适当的自由变量后，它们的函数体类型定义相等。

```
defEq (Pi s a) (Pi t b)
  if defEq s.type t.type
  then
    let thisFvar := fvar s
    defEq (inst a thisFvar) (inst b thisFvar)
  else
    false
```

Lambda

Lambda 使用与 Pi 相同的测试：

```
defEq (Lambda s a) (Lambda t b)
  if defEq s.type t.type
  then
    let thisFvar := fvar s
    defEq (inst a thisFvar) (inst b thisFvar)
  else
    false
```

结构 eta

若两个元素 `x` 和 `y` 都是某个结构类型的实例，并且字段按照下面的过程定义相等，Lean 就承认它们定义相等。该过程比较一边的构造子实参与另一边的投影字段：

```
defEqEtaStruct x y:
  let (yF, yArgs) := unfoldApps y
  if
    yF is a constructor for an inductive type `T`
    && `T` can be a struct
    && yArgs.len == T.numParams + T.numFields
    && defEq (infer x) (infer y)
  then
    forall i in 0..t.numFields, defEq Proj(i+T.numParams, x) yArgs[i+T.numParams]

-- 我们把 `T.numParams` 加到索引上，
-- 因为只想测试非参数实参。
-- 参数已经因推断类型定义相等而已知 defEq。
```

更普通的同余情形 $T.mk\ a \ \dots\ N = T.mk\ x \ \dots\ M$ ，其中 $[a, \dots, N] = [x, \dots, M]$ ，则直接由 `App` 测试处理。

类 unit 相等

Lean 在下列条件下承认两个元素 $x: S \ p_0 \dots p_N$ 和 $y: T \ p_0 \dots p_M$ 定义相等：

- S 是一个归纳类型；
- S 没有指标；
- S 只有一个构造子，且该构造子除 S 的参数 $p_0 \dots p_N$ 外不接受任何实参；
- 类型 $S \ p_0 \dots p_N$ 与 $T \ p_0 \dots p_M$ 定义相等。

直观地说，这种定义相等是合理的，因为这些类型的元素所能传达的所有信息都已由它们的类型捕获，而我们要求这些类型定义相等。

Eta 展开

```
defEqEtaExpansion x y : bool :=
  match x, (whnf $ infer y) with
  | Lambda .., Pi binder _ => defEq x (App (Lambda binder (Var 0)) y)
  | _, _ => false
```

右侧创建的 lambda，即 $(\text{fun } _ \Rightarrow \$0) \ y$ ，会平凡地规约为 y ；但加入 lambda 绑定子让 x 和 y' 有机会同定义相等过程的其余部分匹配。

证明无关相等

Lean 把证明无关相等视为定义相等。例如，Lean 的定义相等过程会把 $2 + 2 = 4$ 的任意两个证明都视为定义相等的表达式。

若某个类型 T 推断为 $\text{Sort } 0$ ，我们就知道它是一个证明，因为它是 Prop 的一个元素（记住， Prop 是 $\text{Sort } 0$ ）。

```
defEqByProofIrrelevance p q :
  infer(p) == S  $\wedge$ 
  infer(q) == T  $\wedge$ 
  infer(S) == Sort(0)  $\wedge$ 
  infer(T) == Sort(0)  $\wedge$ 
  defEq(S, T)
```

如果 p 是类型 A 的证明， q 是类型 B 的证明，那么只要 A 与 B 定义相等， p 与 q 就因证明无关性而定义相等。

自然数（自然数字面量）

两个自然数字面量定义相等，当且仅当它们可以被规约为 `Nat.zero` 和/或 `NatLit 0`；或者它们可以被规约为 `Nat.succ x`（或 `NatLit x+1`）以及 `Nat.succ y`（或 `NatLit y+1`），其中 `x` 和 `y` 定义相等。

我们预期多数实现会把相同的自然数字面量视为语法相等。也就是说，当 `n` 和 `m` 的值相等时，实现语言会原生地认为 `NatLit n` 与 `NatLit m` 相等。

实践中，这个检查只会在语法相等测试之后执行；在后继情形中，每次对 `defEq` 的递归调用也都应以语法相等检查开始。

```
def isNatZero n:
  match n with
  | Nat.zero | NatLit 0 => true
  | _ => false

def isNatSuccOf n:
  match n with
  | .app Nat.succ n' => some n'
  | NatLit n+1 => some (NatLit n)
  | _ => none

def defEqNatLit a b:
  if isNatZero a && isNatZero b
  then some true
  else
    match isNatSuccOf a, isNatSuccOf b with
    | some a', some b' => some (defEq a' b')
    | _, _ => none
```

字符串字面量

```
StringLit(s), App(String.mk, a)
```

字符串字面量 `s` 会被转换为 `Const(String.mk, [])` 对某个 `List Char` 的应用。由于 Lean 的 `Char` 类型用于表示 Unicode 标量值，这些值的整数表示是 32 位无符号整数。

举例来说，字符串字面量 "ok" 使用两个字符，分别对应 32 位无符号整数 111 和 107，因此会被转换为：

```
(String.mk (((List.cons Char) (Char.ofNat [] NatLit(111))) (((List.cons Char) (Char.ofNat
NatLit(107))) (List.nil Char))))
```

惰性 delta 规约与同余

可用的内核实现会在定义相等检查中实现一种“惰性 delta 规约”过程：它使用[可规约性提示](#)惰性地展开定义，并在看起来有希望时检查同余。这比在检查定义相等之前急切地完全规约两个表达式高效得多。

如果有两个表达式 `a` 和 `b`，其中 `a` 是高度为 10 的定义的一个应用，而 `b` 是高度为 12 的定义的一个应用，那么惰性 delta 过程会采取更高效的路线：展开 `b`，试图让它更接近 `a`；而不是把二者都完全展开，或盲目选择一边展开。

如果惰性 delta 过程发现两个表达式都是某个 `const` 表达式对若干实参的应用，并且这些 `const` 表达式指向同一声明，则会检查这些表达式是否同余（即它们是否是同一个 `const` 应用于定义相等的实参）。同余失败会被缓存；对于编写自己内核的读者，缓存这些失败是一项性能关键的优化，因为同余检查涉及一次可能代价高昂的 `def_eq_args` 调用。

规约

规约的目的，是把表达式推向其**范式**，从而判断表达式是否定义相等。例如，为了判断 `(fun x => x) Nat.zero` 与 `Nat.zero` 定义相等，我们需要执行 beta 规约；为了判断 `id List.nil` 与 `List.nil` 定义相等，我们需要执行 delta 规约。

Lean 内核中的规约有两个性质，会引出一些在基础教材处理中有时未被讨论的问题。第一，在某些情况下，规约与推断是交错进行的。除其他事情外，这意味着规约可能需要在打开项上执行，尽管规约过程本身并不创建自由变量。第二，应用于多个实参的 `const` 表达式，在规约期间可能需要同这些实参一起考虑（如 `iota` 规约中那样），因此在规约开始时，需要一起展开一串应用。

Beta 规约

Beta 规约处理函数应用的规约。具体而言，规约：

```
(fun x => x) a    ~~>    a
```

Beta 规约的实现必须对表达式去脊（despine），以收集 `app` 表达式中的所有实参；检查这些实参所应用到的表达式是否是 `lambda`；然后在函数体中，把对应有界变量的所有出现替换为适当的实参：

```
betaReduce e:
  betaReduceAux e.unfoldApps

betaReduceAux f args:
  match f, args with
  | lambda _ body, arg :: rest => betaReduceAux (inst body arg) rest
  | _, _ => foldApps f args
```

对于 beta 规约的实例化（替换）部分，一个重要的性能优化有时被称为“广义 beta 规约”。它会收集那些有对应 `lambda` 的实参，并一次性替换它们。这个优化意味着，对于 `n` 个连续的、施加了实参的 `lambda` 表达式，我们只需遍历表达式一次来替换相应实参，而不是遍历 `n` 次。

```
betaReduce e:
  betaReduceAux e.unfoldApps []

betaReduceAux f remArgs argsToApply:
  match f, remArgs with
  | lambda _ body, arg :: rest => betaReduceAux body rest (arg :: argsToApply)
  | _, _ => foldApps (inst f argsToApply) remArgs
```

Zeta 规约（规约 let 表达式）

Zeta 规约是 `let` 表达式规约的一个较华丽名称。具体而言，规约

```
let (x : T) := y; x + 1    ~~>    (y : T) + 1
```

实现可以简单到如下程度：

```
reduce Let _ val body:
  instantiate body val
```

Delta 规约（定义展开）

Delta 规约指展开定义（以及定理）。Delta 规约会把一个 `const ..` 表达式替换为被引用声明的值，同时把该声明的泛型宇宙参数替换为当前上下文相关的宇宙参数。

如果当前环境包含一个定义 `x`，它以宇宙参数 `u*` 和值 `v` 声明，那么可以对表达式 `Const(x, w*)` 作 delta 规约：先把它替换为 `val`，再把宇宙参数 `u*` 替换为 `w*` 中的那些层级。

```
deltaReduce Const name levels:
  if environment[name] == (d : Declar) && d.val == v then
    substituteLevels (e := v) (ks := d.uparams) (vs := levels)
```

如果为了到达被 delta 规约的 `const` 表达式而不得不移除某些已施加实参，那么这些实参应当重新应用到规约后的定义上。

投影规约

`proj` 表达式有一个自然数指标，表示要投影的字段；还有另一个表达式，表示结构本身。构成该结构的实际表达式，应当是一串实参应用到某个引用构造子的 `const` 上。

请记住，对于完全 elaborated 的项，传给构造子的实参会包含任何参数。因此，`Prod` 构造子的一个实例会形如 `Prod.mk A B (a : A) (b : B)`。

表示被投影字段的自然数从 0 开始计数，其中 0 是构造子的第一个非参数实参，因为投影不能用于访问结构的参数。

有了这一点就很清楚：一旦我们把构造子的实参去脊为 `(constructor, [arg0, .., argN])`，就可以简单地取位置 `i + num_params` 处的实参来规约投影，其中 `num_params` 正如其名，是该结构类

型的参数数量。

```
reduce proj fieldIdx structure:
  let (constructorApp, args) := unfoldApps (whnf structure)
  let num_params := environment[constructorApp].num_params
  args[fieldIdx + numParams]

-- 沿用 `Prod` 例子, constructorApp 会是 `Const(Prod.mk, [u, v])`
-- args 会是 `[A, B, a, b]`
```

投影的特殊情形：字符串字面量

字符串字面量的内核扩展在投影规约中引入了一个特殊情形，在 `iota` 规约中也引入了一个特殊情形。

字符串字面量的投影规约：由于投影表达式的结构可能规约为字符串字面量（Lean 的 `String` 类型被定义为一个只有一个字段的结构，该字段是 `List Char`）。

如果该结构规约为 `StringLit (s)`，我们就把它转换为 `String.mk (.. : List Char)`，然后像通常的投影规约一样继续。

Nat 字面量规约

自然数字字面量的内核扩展包括对 `Nat.succ` 的规约，以及加法、减法、乘法、幂、除法、取模、布尔相等和布尔小于等于这些二元操作的规约。

若正在规约的表达式为 `Const(Nat.succ, []) n`，且 `n` 可以规约为自然数字字面量 `n'`，则将其规约为 `NatLit(n'+1)`。

若正在规约的表达式为 `Const(Nat.<binop>, []) x y`，且 `x` 和 `y` 可以规约为自然数字字面量 `x'` 与 `y'`，则将适当 `<binop>` 的宿主语言版本应用于 `x'` 与 `y'`，并返回所得自然数字字面量。

示例：

```
Const(Nat.succ, []) NatLit(100) ~> NatLit(100+1)

Const(Nat.add, []) NatLit(2) NatLit(3) ~> NatLit(2+3)

Const(Nat.add, []) (Const Nat.succ [], NatLit(10)) NatLit(3) ~> NatLit(11+3)
```

Iota 规约（模式匹配）

Iota 规约是指应用特定于某个归纳声明、并由该归纳声明派生出的规约策略。这里说的是对归纳声明递归器的应用（或者我们稍后会看到的 `Quot` 特殊情形）。

每个递归器都有一组“递归器规则”，每个构造子对应一条递归器规则。与呈现为类型的递归器不同，这些递归器规则是值层级表达式，展示如何消去一个由构造子 `T.c` 创建的类型 `T` 的元素。例如，`Nat.rec` 有一条用于 `Nat.zero` 的递归器规则，还有一条用于 `Nat.succ` 的递归器规则。

对于归纳声明 `T`，`T` 的递归器所要求的元素之一是一个实际的 `(t : T)`，也就是我们正在消去的东西。这个 `(t : T)` 实参称为“主前提”。Iota 规约通过拆开主前提，查看用哪个构造子创建了 `t`，再从环境中取出并应用相应的递归器规则，从而执行模式匹配。

由于递归器的类型签名也要求所需的参数、动机和次前提，因此我们不需要为了在例如 `Nat.zero` 而非 `Nat.succ` 上执行规约而改变递归器的实参。

在实践中，有时需要先做一些初始操作，以暴露创建主前提所用的构造子，因为它可能不是一个直接的构造子应用。例如，`NatLit(n)` 表达式需要被转换为 `Nat.zero`，或 `App Const(Nat.succ, [])` ..。对于结构，我们也可以执行结构 eta 展开，把元素 `(t : T)` 转换为 `T.mk t.1 .. t.N`，从而暴露 `mk` 构造子的应用，使 Iota 规约得以继续（如果无法弄清主前提由哪个构造子创建，规约就失败）。

List.rec 类型

```
forall
  {α : Type.{u}}
  {motive : (List.{u} α) -> Sort.{u_1}},
  (motive (List.nil.{u} α)) ->
  (forall (head : α) (tail : List.{u} α), (motive tail) -> (motive (List.cons.{u}
α head tail))) -> (forall (t : List.{u} α), motive t)
```

List.nil 递归器规则

```
fun
  (α : Type.{u})
  (motive : (List.{u} α) -> Sort.{u_1})
  (nilCase : motive (List.nil.{u} α))
  (consCase : forall (head : α) (tail : List.{u} α), (motive tail) -> (motive
(List.cons.{u} α head tail))) =>
  nilCase
```

List.cons 递归器规则

```
fun
  (α : Type.{u})
  (motive : (List.{u} α) -> Sort.{u_1})
  (nilCase : motive (List.nil.{u} α))
  (consCase : forall (head : α) (tail : List.{u} α), (motive tail) -> (motive
(List.cons.{u} α head tail)))
  (head : α)
  (tail : List.{u} α) =>
  consCase head tail (List.rec.{u_1, u} α motive nilCase consCase tail)
```

类 k 规约

对于某些归纳类型，即所谓“子单例消去器”，只要我们知道主前提的类型，即使主前提的构造子没有直接暴露，也可以继续执行 iota 规约。例如，当主前提作为自由变量出现时，就可能出现这种情况。这称为类 k 规约；它之所以被允许，是因为子单例消去器的所有元素都是相同的。

要成为子单例消去器，一个归纳声明必须是归纳命题，不能是相互归纳或嵌套归纳；它必须恰好有一个构造子，并且唯一构造子只能接受该类型的参数作为实参（它不能“隐藏”任何未完全由其类型签名捕获的信息）。

例如，类型 `Eq Char 'x'` 的任一元素的值完全由其类型决定，因为该类型的所有元素都是相同的。

如果 iota 规约发现一个主前提是子单例消去器，那么允许把该主前提替换为对该类型构造子的应用，因为这是该自由变量实际可能取到的唯一元素。例如，一个类型为 `Eq Char 'a'` 的自由变量形式主前提，可以被替换为显式构造的 `Eq.refl Char 'a'`。

说到具体机制，如果我们忽略了查找并应用类 k 规约，那么作为子单例消去器的自由变量会无法识别相应的递归器规则，iota 规约会失败，而某些本应成功的转换将不再成功。

Quot 规约; Quot.ind 与 Quot.lift

Quot 引入了两个需要由内核处理的特殊情形：一个用于 Quot.ind，一个用于 Quot.lift。

Quot.ind 与 Quot.lift 都处理把函数 f 应用于实参 $(a : \alpha)$ 的情形，其中 a 是某个 Quot r 的组成部分，而这个 Quot r 由 Quot.mk r a 形成。

为了执行规约，我们需要取出作为 f 元素的那个实参，以及作为 Quot 的那个实参；在后者中可以找到 $(a : \alpha)$ 。然后把函数 f 应用于 a 。最后，重新应用那些属于某个外层表达式、而与 Quot.ind 或 Quot.lift 的调用无关的实参。

由于这只是一个规约步骤，我们依赖别处完成的类型检查阶段来保证整个表达式类型良好。

下面重现了 Quot.ind 和 Quot.mk 的类型签名，并把望远镜元素映射到我们应当在实参中找到的内容。带有 $*$ 的元素是规约中我们关心的部分。

```
Quotient primitive Quot.ind.{u} :  $\forall \{ \alpha : \text{Sort } u \} \{ r : \alpha \rightarrow \alpha \rightarrow \text{Prop} \}$   
  {  $\beta : \text{Quot } r \rightarrow \text{Prop} \}$ ,  $(\forall (a : \alpha), \beta (\text{Quot.mk } r \ a)) \rightarrow \forall (q : \text{Quot } r), \beta \ q$   
  
0  |-> {  $\alpha : \text{Sort } u$  }  
1  |-> {  $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$  }  
2  |-> {  $\beta : \text{Quot } r \rightarrow \text{Prop}$  }  
3* |->  $(\forall (a : \alpha), \beta (\text{Quot.mk } r \ a))$   
4* |->  $(q : \text{Quot } r)$   
...
```

```
Quotient primitive Quot.lift.{u, v} : {  $\alpha : \text{Sort } u$  }  $\rightarrow$   
  {  $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$  }  $\rightarrow$  {  $\beta : \text{Sort } v$  }  $\rightarrow$   $(f : \alpha \rightarrow \beta) \rightarrow$   
   $(\forall (a \ b : \alpha), r \ a \ b \rightarrow f \ a = f \ b) \rightarrow \text{Quot } r \rightarrow \beta$   
  
0  |-> {  $\alpha : \text{Sort } u$  }  
1  |-> {  $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$  }  
2  |-> {  $\beta : \text{Sort } v$  }  
3* |->  $(f : \alpha \rightarrow \beta)$   
4  |->  $(\forall (a \ b : \alpha), r \ a \ b \rightarrow f \ a = f \ b)$   
5* |-> Quot  $r$   
...
```

未来工作与未决问题

文件格式

关于把导出文件格式迁移到 json，有一个开放的 RFC，见[此处](#)；这将是一次主版本变更。

确保 Nat/String 被正确定义

对于如何判定导出文件中 `Nat`、`String` 以及相关内核扩展覆盖的操作的声明，是否以不把导出器纳入可信代码的方式，与扩展所期望的内容相匹配，Lean 社区尚未确定一个特定方案。

类似 `Eq` 和 `Quot` 声明所采用的做法（在类型检查器内部手工定义它们，然后断言二者相同）并不可行，因为 `Nat` 上受支持二元操作的完全 elaborated 项过于复杂。

改进 Pollack 一致性

Lean 4 提供了极其强大的设施，用于定义自定义语法、宏以及漂亮打印器行为；Lean 4 内部几乎所有方面也都对用户开放。Lean 设计中的这些元素，是对 Lean 3 时期 mathlib 社区真实反馈的有效回应。

虽然这些特性是 Lean 作为支持大规模形式化工作的工具而获得成功的重要因素，但它们也与 Lean4 的 Pollack 一致性¹，或者说与其缺失，存在张力。如果不在漂亮打印器中复制宏和语法扩展能力，类型检查器就不能稳定地把项以用户可识别的形式读回给用户。然而，把这些特性加入漂亮打印器，会不太令人满意地扩大可信代码基。另一种做法是放弃漂亮打印器，转而使用可信解析器（类似 `metamath zero`），但 Lean 的解析器可以在用户空间中通过自定义语法声明即时修改。

随着 Lean 日趋成熟并得到更多采用，可能会出现一种推动力，促使人们发展技术和实践，使用户能够利用 Lean 的可扩展性，同时牺牲尽可能少的 Pollack 一致性。

前向推理

现有类型检查器实现的是某种形式的后向推理；另一种类型检查策略是接受并检查由外部程序给出的前向推理链，这可能允许类型检查器更加简单。

¹ Freek Wiedijk. Pollack-inconsistency. Electronic Notes in Theoretical Computer Science, 285:85-100, 2012

延伸阅读

- Mario Carneiro 关于 Lean 类型论的[博士论文](#)。
- Lean 官方内核：<https://github.com/leanprover/lean4/tree/master/src/kernel>
- [lean4lean](#)，用 Lean4 重新实现的 Lean 内核。
- [lean4export](#)，Lean 4 环境推荐/官方导出器。
- [lean4checker](#)
- Peter Dybjer 对 Lean 所实现归纳类型的原始描述：P. Dybjer. Inductive families. Formal aspects of computing, 6(4):440-465, 1994.
- Conor McBride 与 James McKinna 的论文 *I am not a number: I am a free variable*，描述了处理有界/自由变量的局部无名方法。
- 来自 *Counterexamples in Type Systems* 的关于归纳类型中非正出现的信息：
<https://counterexamples.org/strict-positivity.html?highlight=posi#positivity-strict-and-otherwise>
- Freek Wiedijk. Pollack-inconsistency. Electronic Notes in Theoretical Computer Science, 285:85-100, 2012.